# Lab 6: Reinforcement Learning
# Q-Learning Implementation Report

Engin S. Demirkan Franziska Axmann

15 05 2026

## 1 Task Description

The objective of this laboratory exercise is to implement the Q-Learning algorithm from scratch to solve a Reinforcement Learning problem. This report details the solution for Variant 4, utilizing the `Taxi-v4` environment from the Gymnasium library. The implementation is written entirely in Python without the use of dedicated machine learning or optimization libraries, relying solely on NumPy for matrix operations and Matplotlib for visualization.

## 2 Environment Description

*Note: While the assignment specified the `Taxi-v3` environment, modern versions of the Gymnasium library have deprecated and removed it. Therefore, `Taxi-v4` was used for this implementation. It shares the exact same state space, action space, and reward structure as v3, ensuring the learning objectives are perfectly met while maintaining compatibility with modern Python environments.*

The `Taxi-v4` environment is a grid-world task where the agent acts as a taxi driver.

- **Goal:** The objective is to pick up a passenger from one specific location and drop them off at a designated destination as quickly as possible.
- **State Space:** The state space is discrete and consists of 500 possible states. These states are formulated by combining the taxi's location (5x5 grid), the passenger's current location (4 distinct starting coordinates plus 1 state representing "in taxi"), and the destination location (4 coordinates).
- **Action Space:** The action space is discrete with 6 possible actions: Move South (0), Move North (1), Move East (2), Move West (3), Pick up passenger (4), and Drop off passenger (5).
- **Rewards:**
  - -1 for every step taken (to encourage finding the fastest route).
  - +20 for successfully dropping off the passenger at the correct destination.
  - -10 for illegally attempting to pick up or drop off a passenger.

- **Terminal Conditions:** An episode terminates naturally when the passenger is successfully dropped off at the destination. It is additionally truncated if the step limit (200 steps) is reached.

Because the observation space is completely discrete, no quantization method was necessary to represent the states within the Q-table.

# 3 Experiments and Results

## 3.1 Experiment Setup

To understand the algorithm's performance, an experiment was conducted testing the impact of the learning rate ($\alpha$) on the convergence rate and total reward accumulation.

- **Measured Metric:** Total reward per episode (smoothed using a 100-episode moving average to better visualize trends).
- **Fixed Hyperparameters:** Number of episodes = 2000, Discount factor ($\gamma$) = 0.99, Initial exploration rate ($\epsilon$) = 1.0, Exploration decay rate = 0.995, Minimum exploration rate = 0.01.
- **Tested Hyperparameters:** Learning rate ($\alpha$) values of 0.1, 0.5, and 0.9.

## 3.2 Experiment Results

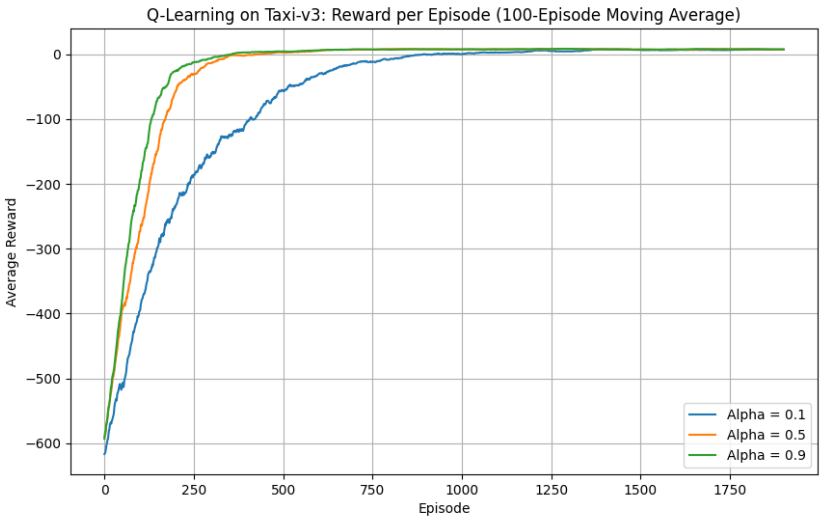


Figure 1: 100-Episode Moving Average of Rewards for Different Learning Rates on Taxi-v4.

## 3.3 Results Discussion

The training plots showcase how different learning rates alter the convergence of the Q-Learning algorithm in the `Taxi` environment.

- $\alpha$ = **0.9 (High Learning Rate):** The agent aggressively updates its Q-values based on the most recent observations. In a deterministic environment like Taxi, this allows for rapid initial learning, reaching the optimal policy (averaging positive rewards) in roughly 250 episodes.
- $\alpha$ = **0.5 (Moderate Learning Rate):** This provides a balanced approach, converging slightly slower than 0.9 but eventually matching its stable maximum reward trajectory without over-correcting.
- $\alpha$ = **0.1 (Low Learning Rate):** A lower learning rate leads to significantly slower convergence. The agent takes much longer to overwrite its initial Q-value estimates, spending over 1000 episodes exploring suboptimal paths before finally stabilizing near the maximum reward.

Ultimately, the algorithm successfully learned the optimal policy to navigate the grid, pick up the passenger, and complete the drop-off efficiently under all tested learning rates, though the speed of convergence varied drastically based on the selected $\alpha$.